

OpenCollab + GLM-5.2

SWE-bench Lite 本地评测与 Team 模式实验总结

实验整理

2026-06-28

摘要

本报告汇总了 OpenCollab 接入 GLM-5.2 后，在 SWE-bench Lite 前 100 题上的一系列本地实验，并纳入后续 Team 模式、空 patch 复跑、temperature=0.2 与 temperature=0.0 对照实验的官方评测结果。

1 实验对象与口径

实验对象是 SWE-bench Lite 的前 100 个实例。patch 全部由本地 OpenCollab 仓库配合 GLM-5.2 生成，没有用任何线上服务。是否通过一律以 SWE-bench harness 给出的 `report.json` 为准，标准就是 `resolved=true`。

2 单 Agent 前 100 题结果

第一轮跑下来，100 题里有 81 题给出了非空 patch，官方评测过了 50 题、挂了 31 题。剩下 19 题最终 patch 为空。这 19 题我们没有直接放弃，而是做了监测复跑、24 步复查，并修好了 `django__django-14238` 这道题损坏的镜像，之后它们全部拿到了官方判定，通过 6 题、未通过 13 题。两部分合起来，前 100 题目前可审计的状态是 56 通过、44 未通过。

阶段	样本数	通过	未通过	通过率
初始非空 patch	81	50	31	61.7%
初始空 patch 后续补齐	19	6	13	31.6%
单 Agent 当前可审计状态	100	56	44	56.0%

表 1: 单 Agent 前 100 题当前官方判定

值得一提的是，那 31 道非空但未通过的题，patch 本身都能干净地 apply，问题出在目标测试还是不过、或者改出了回归。后来补齐的 19 道空 patch 题也都顺利进了官方评测。这两点放在一起其实说明，patch 的格式或应用已经退居次要位置，往后更该关注模型的编辑策略、写入是否可靠、步数怎么控制，以及镜像本身健不健康。

3 空 patch 研究

按旧记录，这 19 个空 patch 大致能分成三类。一类是改过但最终 diff 为空，一类是改到一半卡住或自己撤回，还有一类从头到尾压根没写入过有效内容。它们麻烦的地方都一样——最终 patch 字符串是空的，光看产物根本没法判断中间到底发生了什么。为此我们在复跑里加了更细的埋点，把每次文件写入的结果、最终 diff、循环检测、工具报错和 Docker 镜像状态都记了下来。

旧空 patch 类别	数量	12 步复跑后非空
曾改动但最终 diff 为空	5	5
改动后卡住或撤回	6	6
无有效写入	8	4

表 2: 初始空 patch 的旧分类与复跑结果

最有意思的证据来自 12 步复跑后仍然异常的那 4 个样本。django__django-11797 和 django__django-15738 只是 12 步之内没来得及真正落笔。django__django-12589 反复读同一个大文件，触发了 loop detector，卡在动手改之前。django__django-14238 竟然是镜像坏了。

复查的结论很清楚，步数上限太短，会把那些“想明白了但还没动手”的会话硬生生截成空 patch。把上限放宽到 24 步后，django__django-11797、django__django-12589 和 django__django-15738 都给出了非空 patch，尽管官方评测仍然没过。django__django-14238 在镜像修好之后生成了一个 651 字符的 patch，并顺利通过。这个个案的教训很直接，镜像健康检查必须放在调用模型之前，坏镜像导致的失败不该算到模型头上。

4 Team 模式结果

组合策略	通过题数	通过率
单 Agent	56/100	56.0%
Team	78/100	78.0%

表 3: 单 Agent 与 Team 模式通过率对比

5 Team 模式失败案例分析

Team 模式没解出来的题，patch 同样都能正常 apply，卡点全在测试。具体看，django__django-13590 和 django__django-13660 是另一种形态——目标测试过了，却把原本通过的 PASS_TO_PASS 改回归了。其余大多数样本则是 FAIL_TO_PASS 还有残留。换句话说，到这一步，Team 模式的失败已经和 patch 能不能用没关系了，纯粹是语义层面的修复还不够准。

实例	F2P 失败	P2P 失败	首个失败信号
astropy__astropy-14365	1	0	QDP roundtrip case handling
astropy__astropy-7746	1	0	zero-size WCS input
django__django-13321	18	0	invalid session data decoding
django__django-11019	13	0	forms media combine/construction
django__django-11283	1	0	auth proxy permission migration
django__django-11564	2	0	SCRIPT_NAME static/media prefix
django__django-11630	2	0	duplicate db table checks
django__django-11742	2	0	choice max length checks
django__django-11848	2	0	RFC 850 date parsing
django__django-11905	2	0	isnull lookup behavior
django__django-12470	1	0	inherited ordering by pk desc
django__django-13590	0	5	iterable lookup expression regressions
django__django-13660	0	2	shell command globals regressions
django__django-13768	1	0	send_robust logging behavior
django__django-14017	1	0	Q object and boolean expression combination
django__django-14155	3	0	ResolverMatch repr for partial functions
django__django-14730	1	0	related_name on symmetrical many-to-many
django__django-15202	2	0	URLField validation
django__django-15213	1	0	full expression annotation aggregation
django__django-15252	2	0	TEST MIGRATE=False schema behavior
django__django-15695	1	0	unnamed index rename migration
django__django-15738	1	0	FK to M2M migration ordering

6 失败模式归因

为了搞清楚这 22 题到底卡在哪, 我们把两批官方评测 (Team 34 题里的 19 个未解 + Team 10 题里的 3 个未解, 正好凑成 22) 的 `patch.diff` 和 `test_output.txt` 都翻了一遍, 并抽查了覆盖各种失败规模的样本——大面积挂的 `django__django-11019` 和 `django__django-13321`、引入 P2P 回归的 `django__django-13590` 和 `django__django-13660`、只差一两个用例的 `django__django-14155` 和 `django__django-14017`, 以及两道 `astropy`。结论相当集中。这 22 题没有一道是 `patch` 用不了的, 全部成功 `apply`。失败几乎都是同一回事, 模型找对了文件、也改在了正确的函数上, 但语义修复差了一口气。换句话说, 到这一步工程和格式已经退居次要位置, 核心问题变成了模型对 bug 根因的理解还不够精确。

具体可以拆成三类。第一类是改对了位置、思路也接近, 但实现写错了。`django__django-13590`

把 `type(value)(...)` 改成 `type(value)(*[...])`，可 `tuple()/list()` 只收一个参数，直接报 `TypeError`，还把原本通过的 5 个 P2P 测试带崩了。django__django-14017 给 `Q._combine` 加了 `Q(other)` 包装，但方式不对，跑出 `'Exists' object is not subscriptable`。django__django-11019 把 `media merge` 整个重写成拓扑排序，方向其实是对的，但依赖图实现有 bug，排序结果错，13 个 F2P 全挂。django__django-13321 在 `_legacy_decode` 里只 catch 了 `binascii.Error`，改动范围太窄，和正解的整体重构对不上，18 个 session 测试挂掉。

第二类是只实现了一半，漏改了配套的另一处。django__django-14155 加了 `functools.partial` 的解包逻辑，却没同步改 `__repr__`，测试期望的带引号 `url_name` 显示没出来。django__django-13660 知道 `exec` 要传命名空间，把 `exec(cmd)` 改成了 `exec(cmd, {})`，但传的是空 dict，丢了 `globals`，命令里的 `import` 全失效，断言 `'False' != 'True'`。astropy__astropy-14365 给读取正则加了 `re.IGNORECASE`（这正是正解），可只修了读的路径、没修写出路径，于是 `roundtrip` 的大小写对不上，还顺手在仓库根目录留了个垃圾 `test.qdp`。astropy__astropy-7746 加了零尺寸输入的处理，但边界返回的形状不全，差一个 F2P。

第三类数量最多，是差一两个边界用例的近失。django__django-11283、-11564、-11630、-11742、-11848、-11905、-12470、-13768、-14730、-15202、-15213、-15252、-15695、-15738 都属于这一档——F2P 只挂 1 到 3 个、P2P 全过，patch 通常也就十几到三十行，主逻辑改对了，只是某个特定 case（空值、命名约定、迁移顺序之类）没覆盖到。

失败模式	题目数	代表样本
实现写错（思路接近）	4	django__django-13590、-14017、-11019、-13321
只改了一半	4	django__django-14155、-13660、astropy__astropy-14365、astropy__astropy-7746
差边界用例的近失	14	django__django-11742、-11848、-15695 等

有两点值得单独标注。其一，真正引入 P2P 回归的只有 django__django-13590 和 django__django-13660 两题，也就是说“改对了目标却把别处弄坏”的情况很少，绝大多数失败是目标测试本身还没全过——这说明当前 GLM-5.2 整体偏保守，乱改情况较少。其二，astropy__astropy-14365 的测试日志里混着 `numpy C 扩展的编译报错 (expected PyArrayObject*)`，而它的修复思路其实就是正解，所以这道题的失败到底是逻辑差一截（没改写出路径）还是扩展重编译的环境噪声，值得单独重跑确认，不宜直接算成纯模型失败。

7 重复行为与温度设置

`run_team_selected_batch.py` 和三份 `recovery runner` 里都写着 `OPENCOLLAB_TEMPERATURE=1.0`，构造 `SpawnConfig` 时也是 `temperature=config.get("temperature", 1.0)`。空 patch 复跑那三份 `env_snapshot.json` 同样记着 `args.temperature=1.0` 和 `OPENCOLLAB_TEMPERATURE=1.0`。另外，`OPENCOLLAB_THINKING=false` 也一并写进了环境。

我们对 `sessions/*/agent*.json` 做了去空白后的精确比对，凡是长度超过 80 字符的 assistant 输出，没有一条是完全重复的。Kimi 旧日志里那些 `PROSE-STOP`、`unknown_tool`、`loop_blocked` 信号，这次也一个没见到。

工具层面倒是有一些重复，但都很局部。Team 34 的事件日志里一共 15 条 `loop_detected`，散在 7 个文件中，基本都是同一个文件被 `file_read` 读了 8 遍以上，典型的有 `django__django-13265`、`django__django-14730`、`django__django-15738` 和 `django__django-13590`。真正的精确重复调用只集中在两处。`django__django-13590` 的 tester 把 `django/db/models/sql/query.py` 同一个 22 行窗口读了 5 次，`django__django-14667` 的 tester 把同一个目标测试跑了 3 遍。空 patch 复跑里也有类似苗头——`django__django-11019` 命中过重复 `file_write`，`django__django-12589` 和 `django__django-15738` 命中过重复读同一文件。总的看，这批 GLM-5.2 有少量工具重复和局部原地打转，但没有出现 Kimi 旧实验那种长时间复述同一段输出的硬死循环。

8 低温对照实验 (temperature=0.2)

前面的运行都用 `temperature=1.0`。为了看看降温能不能把那些”思路对、实现差一口气”的题救回来，随机挑了四道题做试验。

题目	官方结果	主要情况
<code>astropy__astropy-7746</code>	未通过	仍失败 <code>test_zero_size_input</code>
<code>django__django-11019</code>	未通过	14 个 F2P 失败，生成阶段出现 <code>loop_detected</code> 和 <code>budget_exceeded</code>
<code>django__django-13321</code>	未通过	18 个 legacy decode 相关 F2P 仍失败
<code>django__django-13590</code>	通过	修掉了 <code>namedtuple range lookup</code> ，且没有 P2P 回归

表 5: 低温 `temperature=0.2` 前 4 题官方评测

真正有价值的是 `django__django-13590`。`temperature=1.0` 的 Team 版当时是目标测试过了、却带了 P2P 回归（见前文失败模式归因），降到 0.2 之后这次官方评测直接通过了。另外三题没能靠低温救回来。从过程看，0.2 确实更保守，但并不一定更快——`django__django-11019` 反而出现了局部反复加预算耗尽。

9 低温对照实验 (temperature=0.0)

最后一组实验把 Team 模式的温度降到 0.0，对象是 Team `temperature=1.0` 仍未解的 22 题。生成阶段 22 题都留下了 metrics，成功记录合计 6,535,787 tokens；第 22 题 `django__django-15738` 先经历一次掉盘前失败尝试，已知额外 token 为 504,236，重连后又生成了一个 2,367 字节的非空 patch。

过程监测显示，22 题里有 14 题没有明显重复信号，4 题出现 `budget_exceeded`，4 题出现 `loop_detected` 或局部重复工具调用。第 22 题最值得注意：它最终生成了非空 patch，但 patch 只新增了 `repro.py`，没有修改 Django 源码。随后对这 22 个 patch 全部跑了 SWE-bench official eval，patch 全部能 apply。前 21 题补评通过 5 题、未通过 16 题；第 22 题 `django__django-15738` 也未

通过。因此 temperature=0.0 这组最终官方成绩为 5/22。中途 `django__django-14155` 第一次评测因为 `raw.githubusercontent.com` 的 SSL EOF 下载错误失败，单题重跑后拿到有效官方判定。

实例	token	步数	patch	生成期信号
<code>astropy__astropy-14365</code>	61,237	5	617	无明显重复信号
<code>astropy__astropy-7746</code>	136,083	7	1,106	无明显重复信号
<code>django__django-13321</code>	176,933	8	720	无明显重复信号
<code>django__django-11019</code>	514,364	12	5,347	budget_exceeded
<code>django__django-11283</code>	270,749	13	921	run_tests 重复
<code>django__django-11564</code>	272,169	17	727	无明显重复信号
<code>django__django-11630</code>	503,970	19	1,885	无明显重复信号
<code>django__django-11742</code>	305,510	19	1,578	无明显重复信号
<code>django__django-11848</code>	104,304	8	920	无明显重复信号
<code>django__django-11905</code>	197,149	8	938	无明显重复信号
<code>django__django-12470</code>	194,617	22	816	无明显重复信号
<code>django__django-13590</code>	372,419	6	941	bash 重复
<code>django__django-13660</code>	237,894	9	1,600	run_tests 重复
<code>django__django-13768</code>	93,376	11	995	无明显重复信号
<code>django__django-14017</code>	508,882	8	1,267	budget_exceeded
<code>django__django-14155</code>	228,686	12	783	无明显重复信号
<code>django__django-14730</code>	260,514	13	1,036	无明显重复信号
<code>django__django-15202</code>	187,143	5	1,460	无明显重复信号
<code>django__django-15213</code>	501,525	16	2,042	budget_exceeded
<code>django__django-15252</code>	508,354	20	2,064	budget_exceeded
<code>django__django-15695</code>	395,795	13	1,287	无明显重复信号
<code>django__django-15738</code>	504,114	19	2,367	file_read 重复；官方评测未通过

表 6: temperature=0.0 的 22 题生成 token 与过程信号

实例	结果	F2P	P2P	首个失败信号
astropy__astropy-14365	未过	1	0	QDP roundtrip
astropy__astropy-7746	未过	1	0	zero-size WCS input
django__django-13321	未过	18	0	cookie session legacy decode
django__django-11019	未过	16	4	forms media combine/construction
django__django-11283	未过	1	0	proxy permission migration
django__django-11564	未过	2	0	SCRIPT_NAME static/media prefix
django__django-11630	未过	2	0	duplicate db table checks
django__django-11742	未过	2	0	field choices max length checks
django__django-11848	通过	0	0	–
django__django-11905	未过	2	0	isnull lookup behavior
django__django-12470	未过	1	0	inherited ordering by pk desc
django__django-13590	通过	0	0	–
django__django-13660	通过	0	0	–
django__django-13768	未过	1	0	send_robust failure logging
django__django-14017	通过	0	0	–
django__django-14155	未过	3	0	ResolverMatch repr
django__django-14730	未过	1	0	useless related_name on M2M
django__django-15202	未过	2	0	URLField validation
django__django-15213	通过	0	0	–
django__django-15252	未过	2	0	TEST MIGRATE=False schema behavior
django__django-15695	未过	1	0	unnamed index rename migration
django__django-15738	未过	2	0	FK to M2M migration ordering

表 7: temperature=0.0 的 22 题官方评测结果

10 重复句直到预算耗尽的取证

重新抽取 `events.jsonl` 后，可以确认上一轮确实出现了一次非常典型的“同一句判断反复输出，直到预算耗尽”的模式。最核心的个案是 temperature=0.0 的 `django__django-15738`。在第 468 到 572 行之间，coder 连续 25 次重复同一个核心判断：The edit was already applied successfully earlier。其中 16 次还带着同一段解释：The bash/file_read errors are due to a docker socket wrapper issue in this environment；21 次带着同一个动作计划：Let me verify the file state and run tests using a different approach。这段之后，事件日志第 575 行记录 coder budget_exceeded，第 578 行记录 lead budget_exceeded。同一题后续重启后又在第 842 和 845 行再次触发预算耗尽。

这个模式的关键不在于模型完全不知道自己在重复。它每次都认为“编辑已经成功，工具报错只是环境问题，所以再换一种方式验证”。问题是，每一次“换一种方式”实际仍然回到同一个验证动作附近，loop detector 也连续报出 bash 重复，计数从 3 增到 18。也就是说，这里出现的是**自我确认被工具失败放大**：模型把一次 `str_replace` 的成功信号当作锚点，随后把所有读文件和跑测试失败都解释成环境噪声，于是不断尝试证明同一个结论，最后耗光预算。

实例	温度	重复证据	后果
django__django-15738	0.0	edit-applied 核心句 25 次; docker-wrapper 解释 16 次; different-approach 计划 21 次; 事件日志 468-572; bash loop count 3-18。	第 575/578 行预算耗尽; 重启后 842/845 行再次预算耗尽。已知失败尝试 504,236 tokens, 重试 504,114 tokens。最终 patch 非空, 但只带出 repro.py, 官方评测未通过。
django__django-13590	0.0	list/tuple 句 17 次; handle-both 句 10 次; upstream-fix 句 10 次; stop-looping 自觉 3 次。	触发 9 条 loop detector, 但没有预算耗尽; 最终跳出循环并生成 patch。这个样本说明 loop detector 能发现问题, 却没有强制改写策略。
django__django-11019	0.0	Docker connectivity 说法 3 次; retry 说法 6 次。	多个 agent 相继预算耗尽, 第 175/267/270/418/455 行出现 budget_exceeded。这里以 Docker/路径失败引发的反复重试为主, 纯文本复读成分较少。

表 8: temperature=0.0 中与重复输出和预算耗尽相关的主要证据

这三个样本合在一起说明, temperature=0.0 确实更容易让 agent 固着在一个确定性解释上。它不一定表现为旧 Kimi 那种一整段自然语言无意义复述; 更多时候是“同一个判断句 + 同一种工具尝试”循环出现。15738 是最严重版本, 重复句、loop detector 和预算耗尽三者完全重合。13590 是中间状态, 模型已经意识到自己在循环, 但 loop detector 只记录信号, 没有把它强制切换成新的行动策略。11019 则提醒我们, 外部 Docker 或文件访问错误也会诱发相似的重试行为, 不能把所有重复都归为模型语义能力问题。

对 harness 来说, 单纯记录 loop_detected 还不够。更合适的处理是: 当同一 agent 在短窗口内重复同一自然语言判断超过 3 次, 同时工具 loop count 继续上升, 就应该强制进入“停止验证、保存当前 diff、换角色复核”的分支。若重复句里含有“edit was already applied”“environment issue”“retry”等关键词, 还应该把最近一次成功写入、最近一次 git diff、最近三次工具错误和当前 patch 一并保存。这样可以避免 15738 这种情况继续烧 token, 也能保留足够证据判断到底是工具层故障、模型误判, 还是 patch 提取出了问题。

11 对 OpenCollab 运行策略的建议

写入工具得留下更完整的审计痕迹。每次写入之后, 把写前 hash、写后 hash、文件大小变化、git diff --stat 和写入时的 stderr 都存下来。一旦出现“中途明明写成功了、最终 patch 却是空的”这种情况, 系统应该自动把工作区 diff、staged diff、最后几条模型消息和工具调用摘要一并保存。有了这些, 下回就不用靠人去翻几千行日志, 才能分清到底是模型自己撤回、工具失败, 还是最后提取环节出了问题。

评测侧还有一个小坑。run_swebench_eval_per_instance.py 现在只把 timeout 传给了 SWE-bench harness, 外层的 subprocess.run() 自己却没有 timeout。建议给外层补一个 watchdog, 免得 harness 在资源清理阶段悄无声息地干等下去。

12 1M token 预算复跑

前面 temperature=0.0 的实验里，有 4 题触发 `budget_exceeded`，我们把这 4 题单独拿出来，把 `OPENCOLLAB_BUDGET` 调到 1,000,000

这次 4 题全部生成了非空 patch，官方 SWE-bench 评测通过 1 题、未通过 3 题，没有 patch apply failure。`django__django-14017` 在 1M 预算下通过，说明它此前确实存在预算不足或过早停止的影响。`django__django-11019`、`django__django-15213` 和 `django__django-15252` 继续未通过，且失败都来自 F2P 残留；这些样本加预算后仍没解决，说明瓶颈已经更接近语义修复本身。

实例	token	patch 字节	用时 (s)	结果	F2P/P2P	首个失败信号
django__django-11019	706,356	5,475	573	未过	14/0	forms media combine/construction
django__django-14017	441,387	1,224	496	通过	0/0	—
django__django-15213	1,014,086	4,630	802	未过	1/0	full expression annotation with aggregation
django__django-15252	531,489	915	477	未过	2/0	TEST MIGRATE=False schema behavior

表 9: temperature=0.0、1M 预算四题复跑

13 Loop detect 的全局作用评估

为了回答“全量 GLM-5.2 实验里是否有必要监测重复行为，Loop detect 到底有没有发挥关键作用”，我们重新扫描了 `eval_work` 下所有 `*.events.jsonl`。当前可见事件日志共 87 个，其中 12 个文件出现过真实的 `loop_detected`，总计 56 次；13 个文件出现 `error` 事件，其中 4 个同时有 `loop` 事件。这个比例说明重复工具调用虽然没有构成主流失败形态，但已经超过了可以忽略的噪声水平。

OpenCollab 的 loop detector 工作在工具调用层。普通工具的完全相同参数重复 3 次就会拦截；`file_read` 会按路径归一化，允许同一文件被读到第 8 次才拦截；最近调用窗口是 200。触发后，系统会把 `loop_blocked` 写入 trace，向模型返回一条 `You are stuck in a loop` 的工具消息，并发出 `loop_detected` 事件。它会阻止那一次重复工具真正执行，但不会自动终止整个实例。

统计项	数值	含义
事件日志总数	87	本报告阶段可扫描的 OpenCollab 运行日志
含 <code>loop_detected</code> 的日志	12	至少一次重复工具调用被拦截
<code>loop_detected</code> 总次数	56	全部被记录的重复工具拦截事件
含 <code>error</code> 的日志	13	运行期出现错误事件
同时含 <code>error</code> 与 <code>loop</code> 的日志	4	环境/工具错误和重复行为相互放大的样本

最典型的三类证据如下。`django__django-13590` 在 temperature=0.0 的 22 题实验中出现 9 次 `loop` 事件，`bash` 重复计数最高到 10，但后续仍跳出了局部循环，生成 patch，并且官方评测通过。这说

明 loop detector 有时能当作有效的方向盘,把会话从重复工具尝试里推出来。`django__django-15738` 则是反例: 同一轮 `temperature=0.0` 实验里, 它出现 27 次 loop 事件, `file_read` 最高到 11, `bash` 最高到 18, 随后触发 `budget_exceeded` 并没有产出可用修复。这里 loop detector 记录了灾情, 也拦住了重复工具, 但没有强到足以中断烧 token。`django__django-15213` 的 1M 预算复跑显示了另一种盲区: 自然语言判断重复 5 次, 却没有触发工具层 loop, 因为它没有形成足够严格的同工具同参数重复。

因此, 在这批 GLM-5.2 实验里, Loop detect 的结论可以说得比较稳: 通过率主要由语义修复质量决定; 大多数没过的题仍然是语义修复差边界、漏改配套路径或写错实现。可是它确实是全量评测必须保留的护栏。没有它, 大部分正常题可能照样能跑完, 但低温、外置盘不稳定、Docker socket 报错、读文件原地打转这些场景会更容易把预算烧在重复动作上。尤其在 300 题级别运行时, 少数 15738 这种样本足以造成明显 token 浪费和难排查的失败记录。

后续更合理的做法, 是保留现有工具层 loop detector, 同时再加一层实例级监控。当单个实例的 `loop_detected` 超过 5 次, 应在 summary 里标红; 超过 10 次或同一句自然语言判断重复超过 3 次, 应自动保存最近一次成功写入、当前 `git diff`、最近三次工具错误、最近三段 assistant 文本, 并建议停止当前验证动作, 改由另一个角色复核。这样不会误伤偶尔重读文件的正常行为, 也能避免 `django__django-15738` 那种“相信编辑已经成功、反复证明同一件事”的预算消耗。